

Garantias de Pior Caso versus Desempenho Médio de Heurísticas Clássicas para o Bin Packing Unidimensional: um Estudo Experimental

Benjamin Yuji Suzuki ^{*1}

¹ Centro Universitário do Estado do Pará (CESUPA)
Belém – PA – Brazil

benjamim24070067@aluno.cesupa.br

Abstract. *The one-dimensional bin packing problem asks for the minimum number of unit-capacity bins needed to pack a list of items. It is NP-hard, and five classical heuristics — Next Fit, First Fit, Best Fit and their decreasing variants — remain the default choice in practice, backed by decades-old worst-case guarantees: ratio 2 for Next Fit, 1.7 for First/Best Fit, and the tight bound $11/9 \text{ OPT} + 6/9$ for First Fit Decreasing, whose additive constant was only settled in 2007. Theory, however, says little about how far these bounds are from typical behavior. We implement all five heuristics in Rust with zero external dependencies and run a controlled experiment (4 distributions, 5 instance sizes from 1,000 to 16,000 items, 10 repetitions each, 1,000 runs total), measuring both the approximation ratio against the Martello–Toth lower bound L_2 and wall-clock time. Results show the empirical ratios are far below the guarantees (e.g., FFD averages 1.001 on uniform instances against the 1.222 guarantee), that the 3-partition distribution is the hardest for the four fit-based heuristics (the Next Fit instead peaks on the continuous uniform), and that the $O(n^2)$ implementations of FF/BF grow quadratically in practice (fitted exponent $\beta_1 \approx 1.9$), so that the relative advantage of linear-time Next Fit — $706\times$ faster at $n = 16,000$ — widens as n grows, even though the absolute cost of FF at this scale remains in the tens of milliseconds. We also construct adversarial instances that force the worst case, and document a minimal counterexample where sorting strictly hurts First Fit.*

Resumo. *O problema do empacotamento unidimensional (bin packing) pede o número mínimo de recipientes de capacidade unitária necessários para alocar uma lista de itens. Ele é NP-difícil, e cinco heurísticas clássicas — Next Fit, First Fit, Best Fit e suas variantes ordenadas — continuam sendo a escolha padrão na prática, amparadas por garantias de pior caso com décadas: razão 2 para Next Fit, 1.7 para First/Best Fit e o limite exato $11/9 \text{ OPT} + 6/9$ para First Fit Decreasing, cuja constante aditiva só foi fechada em 2007. A teoria, porém, diz pouco sobre quão longe esses limites estão do comportamento típico. Implementamos as cinco heurísticas em Rust, sem dependências externas, e executamos um experimento controlado (4 distribuições, 5 tamanhos de instância de 1.000 a 16.000 itens, 10 repetições, 1.000 execuções no total), medindo tanto a razão de aproximação contra o limite inferior L_2 de Martello–Toth quanto o*

*Estudante de Ciência da Computação — Centro Universitário do Estado do Pará (CESUPA).

tempo de execução. Os resultados mostram que as razões empíricas ficam muito abaixo das garantias (ex.: FFD com média 1.001 em instâncias uniformes, contra a garantia de 1.222), que a distribuição de 3-partição é a mais difícil para as quatro heurísticas baseadas em encaixe (o Next Fit tem seu pior caso na uniforme contínua) e que as implementações $O(n^2)$ de FF/BF crescem de fato quadraticamente (expoente ajustado $\beta_1 \approx 1,9$), de modo que a vantagem relativa do Next Fit linear — $706\times$ mais rápido em $n = 16,000$ — cresce com n , ainda que o custo absoluto do FF permaneça da ordem de dezenas de milissegundos nessa escala. Construímos ainda instâncias adversariais que forçam o pior caso e registramos um contraexemplo mínimo em que ordenar estritamente piora o First Fit.

Palavras-chave: bin packing; heurísticas; análise de aproximação; análise experimental; FFD.

Keywords: bin packing; heuristics; approximation analysis; experimental analysis; FFD.

1. Introdução

Empacotar itens em recipientes é uma das tarefas mais recorrentes da logística e da computação: alocação de arquivos em blocos de disco, tarefas em lotes de processamento, cortes de matéria-prima, e carregamento de cargas em geral. Na sua forma unidimensional, o **problema do empacotamento** (*bin packing*) recebe como entrada uma lista $L = (s_1, s_2, \dots, s_n)$ de n itens com tamanhos $s_i \in (0, 1]$ e devolve como saída uma partição de L em grupos (“bins”) tal que a soma dos tamanhos de cada grupo não exceda 1 e o número de grupos seja mínimo.

O problema é NP-difícil, por redução de 3-PARTIÇÃO (Seção 2). Essa intratabilidade motivou, desde os anos 1970, o estudo de **heurísticas simples**: regras de decisão de uma linha que empacotam cada item assim que ele chega. As cinco clássicas — *Next Fit* (NF), *First Fit* (FF), *Best Fit* (BF), *First Fit Decreasing* (FFD) e *Best Fit Decreasing* (BFD) — têm garantias de pior caso consolidadas: razão 2 para NF, 1.7 para FF/BF e o limite exato $\text{FFD}(I) \leq \frac{11}{9}\text{OPT}(I) + \frac{6}{9}$, cuja constante aditiva correta permaneceu em aberto por 34 anos [Dósa 2007, Dósa et al. 2013].

Essas garantias, porém, respondem apenas à pergunta “quão ruim *pode* ser?”. A pergunta que interessa a quem escolhe um algoritmo na prática é “quão ruim *tipicamente* é?” — e é aí que a literatura clássica é silenciosa. Os resultados probabilísticos existentes (modelo *random-order* de Kenyon [Kenyon 1996], refinados recentemente [Albers et al. 2021, Hebbar et al. 2024, Ayyadevara et al. 2025]) tratam de razões esperadas, mas não comparam o custo temporal das implementações diretas, nem cobrem as distribuições de benchmark usadas na prática [Falkenauer 1996].

Objetivo geral. Quantificar experimentalmente a *distância* entre as garantias de pior caso das cinco heurísticas clássicas e o desempenho que elas efetivamente exibem em instâncias sintéticas de *benchmark*, medindo essa distância em dois eixos — qualidade (razão de aproximação) e custo (tempo de execução) — e investigando quando, e por que, a distância se fecha.

Objetivos específicos. (i) implementar NF, FF, BF, FFD e BFD em Rust com implementações diretas (sem estruturas auxiliares que escondam a complexidade); (ii)

implementar o limite inferior L_2 de Martello–Toth [Martello and Toth 1990] como denominador da razão de aproximação empírica; (iii) executar um experimento fatorial com 4 distribuições $\times$ 5 tamanhos $\times$ 10 repetições; (iv) ajustar a curva empírica de tempo à previsão assintótica e discutir as divergências; e (v) registrar contraexemplos em que a ordenação prévia piora o resultado.

O objetivo geral parte de uma pergunta única e mensurável: *quão longe da garantia de pior caso está, na prática, a heurística que eu vou escolher?* Os cinco objetivos específicos correspondem às cinco etapas que a respondem, e cada uma tem um artefato verificável neste artigo: (i) e (ii) são o código-fonte e o limitador L_2 (Seção 4); (iii) é o protocolo fatorial de 1.000 execuções (Seção 4); (iv) é a regressão log-log reportada na Tabela 3; (v) é o contraexemplo explícito da Seção 4.5, verificado contra o ótimo exato por branch-and-bound.

Justificativa. A principal lacuna identificada na literatura é a ausência de uma avaliação conjunta que relacione a qualidade da aproximação e o tempo de execução de implementações diretas sobre distribuições de *benchmark* consagradas. Este artigo supre essa carência ao fornecer um experimento reproduzível que quantifica, simultaneamente, o rigor da aproximação em relação ao limite inferior ótimo e o custo computacional real associado a essas heurísticas. A contribuição não é atingir as garantias de pior caso, mas *medir o quão longe delas o comportamento típico se encontra* — e mostrar que essa distância é grande o suficiente para reordenar a escolha do algoritmo: na prática, o custo computacional separa as heurísticas muito mais do que a qualidade.

O texto está organizado como segue. A Seção 2 formaliza o problema, apresenta a redução de NP-dificuldade e as garantias teóricas das heurísticas. A Seção 3 discute criticamente a literatura. A Seção 4 detalha a metodologia experimental. Resultados e discussão (2º bimestre) e a conclusão compõem a versão final.

2. Fundamentação Teórica

2.1. Definições e notação

Seja $L = (s_1, \dots, s_n)$ uma lista de itens com $s_i \in (0, 1]$. Uma **solução viável** é uma função $f : \{1..n\} \rightarrow \{1..k\}$ tal que $\sum_{i:f(i)=j} s_i \leq 1$ para todo j . O **custo** de f é k , o número de bins usados. O problema pede a solução viável de custo mínimo; esse mínimo é denotado $\text{OPT}(L)$. A **razão de aproximação** de um algoritmo A é $A(L)/\text{OPT}(L)$; no caso assintótico, $A(L)/\text{OPT}(L)$ quando $\text{OPT}(L) \rightarrow \infty$. Dizemos que A é ρ -*aproximativo* se $A(L) \leq \rho \cdot \text{OPT}(L) + c$ para toda L , com c constante.

2.2. NP-dificuldade

Teorema 1. BIN PACKING (decisão) é NP-completo.

Prova (esboço). Pertence a NP: um certificado é a partição f , verificável em tempo linear. Para a NP-dureza, reduzimos 3-PARTIÇÃO, problema NP-completo no sentido forte [Garey and Johnson 1979]: dados $3m$ inteiros $a_1, \dots, a_{3m}$ com $\sum_i a_i = mB$ e $B/4 < a_i < B/2$, decidir se existe partição em m tripletas de soma exatamente B . Construimos a instância de bin packing com itens $s_i = a_i/B$ (capacidade de bin normalizada para 1) e perguntamos: existe solução com no máximo m bins? Como $s_i > 1/4$,

cabem no máximo 3 itens por bin; como $s_i < 1/2$, dois itens somam estritamente menos que 1, logo todo bin *cheio* tem exatamente 3 itens. A soma total é $\sum s_i = m$, portanto qualquer solução com m bins tem todos os bins com soma exatamente 1, cada um com exatamente 3 itens — ou seja, uma 3-partição. Responder a pergunta de bin packing responde 3-PARTIÇÃO. $\square$

O esboço acima evidencia a natureza intrinsecamente complexa do problema: por ser NP-difícil *no sentido forte*, não se admite a existência de um Esquema de Aproximação Totalmente em Tempo Polinomial (FPTAS), a menos que $P=NP$. É essa característica estrutural que torna a adoção de heurísticas com garantia de aproximação a única via factível para aplicações reais [Williamson and Shmoys 2011].

2.3. As cinco heurísticas e suas garantias

A Tabela 1 resume as heurísticas estudadas. Todas empacotam cada item no momento em que o consideram, sem reabrimento de decisão.

Tabela 1. Heurísticas clássicas e garantias de pior caso.

Heurística	Regra	Tempo (direta)	Garantia
NF	mantém 1 bin aberto; fecha se não couber	$O(n)$	2 OPT
FF	primeiro bin (ordem de abertura) que couber	$O(n^2)$	1.7 OPT
BF	bin com menor resíduo que couber	$O(n^2)$	1.7 OPT
FFD	FF após ordenar decrescente	$O(n^2)$	$\frac{11}{9} \text{ OPT} + \frac{6}{9}$
BFD	BF após ordenar decrescente	$O(n^2)$	$\frac{11}{9} \text{ OPT} + \frac{6}{9}$

As garantias de NF, FF e BF são de [Johnson et al. 1974]. O limite do FFD tem uma história que vale registrar: Johnson provou $11/9 \text{ OPT} + 4$ em 1973, em sua tese; a constante foi sucessivamente reduzida por trabalhos posteriores até Dósa fechar em $6/9$ com prova de *tightness* [Dósa 2007], resultado depois estendido em periódico com a classificação completa por resíduo de $\text{OPT} \bmod 9$ [Dósa et al. 2013].

3. Trabalhos Relacionados

A literatura sobre bin packing é vasta e pode ser organizada em três frentes que dialogam com este trabalho.

Análise de pior caso (linha clássica). Johnson et al. [Johnson et al. 1974] estabelecem os limites de NF, FF e BF e mostram que são justos (*tight*): existem instâncias adversariais que os atingem. Dósa [Dósa 2007, Dósa et al. 2013] fecha o problema da constante aditiva do FFD. Esses resultados delimitam o espaço teórico, mas respondem a uma pergunta que a prática raramente faz: instâncias adversariais construídas para maximizar a razão não se parecem com as instâncias que surgem em aplicações reais.

Análise probabilística (linha do random-order). Kenyon [Kenyon 1996] introduz o modelo em que a ordem de chegada é uma permutação uniforme aleatória e mostra que a razão esperada do BF fica entre 1.08 e 1.5, conjecturando ≈ 1.15 . A conjectura resiste: Albers et al. [Albers et al. 2021] provam $5/4$ no caso especial em que todos os itens excedem $1/3$; Ayyadevara et al. [Ayyadevara et al. 2025] mostram que nesse caso o BF é

exatamente ótimo e resolvem o modelo i.i.d.; Hebbbar et al. [Hebbbar et al. 2024] quebram a barreira de $3/2$ no caso geral, com limite inferior 1.144 obtido computacionalmente. Balogh et al. [Balogh et al. 2021], na frente online, empurram o limite inferior universal para 1.5427. Essa linha é a mais próxima da nossa pergunta — comportamento típico —, mas seus resultados são assintóticos e não incluem o custo temporal das implementações. A ela se soma a análise clássica de caso médio: Coffman et al. [Coffman et al. 1980] provam razão esperada exata $4/3$ para o NF sob $U(0, 1]$; Bentley et al. [Bentley et al. 1984] mostram que o FF é assintoticamente ótimo na expectativa e que o FFD tem excesso esperado $O(1)$ para itens limitados a $1/2$ — resultados considerados inesperados à época, e que nosso experimento confirma empiricamente.

Metaheurísticas e benchmarks (linha experimental). Falkenauer [Falkenauer 1996] propõe algoritmo genético especializado em agrupamento e, no processo, define as distribuições de instâncias que se tornaram o benchmark de fato da área (ex.: $U\{1/10, \dots, 1/2\}$, média 0.275). Martello e Toth [Martello and Toth 1990] contribuem os limites inferiores L_1/L_2 e o solver exato MTP. Karmarkar e Karp [Karmarkar and Karp 1982] mostram que a aproximação quase-exata é possível em teoria ($\text{OPT} + O(\log^2 \text{OPT})$), com constantes inviáveis na prática. Nenhum desses trabalhos, contudo, reporta o custo temporal das implementações *diretas* (sem árvore de busca) das heurísticas clássicas em escala — o gap que nosso experimento mede.

Síntese crítica. As três frentes cobrem garantia (o quão ruim pode ser), expectativa (o quão ruim é em média, assintoticamente) e instâncias (como gerar benchmarks). Falta o elo: medir, lado a lado, qualidade e tempo das implementações diretas nas distribuições de benchmark. É o que fazemos a seguir.

4. Metodologia

4.1. Implementação

As cinco heurísticas e o lower bound L_2 foram implementadas em Rust (edição 2021, compilador 1.98), em implementações *diretas*: FF/BF percorrem a lista de bins abertos linearmente, sem árvore de segmentos nem estrutura auxiliar — a complexidade medida é a complexidade das estruturas escolhidas. As únicas dependências externas são `rand` (geração de instâncias com semente) e `clap` (interface de linha de comando). O código, o gerador de instâncias e os scripts de experimento estão disponíveis em repositório público¹ com instruções completas de reprodução.

4.2. Instâncias

Quatro distribuições de tamanhos de itens, todas com suporte em $(0, 1]$:

- **uniforme_continua:** $U[0, 1]$;
- **uniforme_discreta_100:** $U\{1/100, \dots, 1\}$;
- **tres_particao:** $U\{0.26, \dots, 0.50\}$ — todos os itens em $(1/4, 1/2]$, o caso 3-partição, deliberadamente difícil [Ayyadevara et al. 2025];
- **falkenauer_u120:** $U\{1/10, \dots, 1/2\}$ (média 0.275) — a distribuição clássica de benchmark [Falkenauer 1996].

¹<https://github.com/Benjamin-Yuji-Suzuki/bin-packing-heuristics>

Justificativa do gerador: distribuições uniformes são o padrão da literatura experimental da área desde [Falkenauer 1996]; a 3-partição isola o efeito de instâncias sem itens pequenos (que serviriam de “argamassa”); a Falkenauer replica o benchmark histórico. Sementes determinísticas (misturadas com n via `splitmix64`) garantem reprodutibilidade total.

4.3. Métricas e protocolo

Para cada instância I : (i) qualidade — $A(I)/L_2(I)$, onde L_2 é o limite inferior de Martello–Toth [Martello and Toth 1990], calculado uma única vez por instância; como $L_2(I) \leq \text{OPT}(I)$, tem-se $A(I)/L_2(I) \geq A(I)/\text{OPT}(I)$ — as razões reportadas são conservadoras (a qualidade real é ainda melhor); (ii) tempo — relógio de parede em microsegundos (`Instant::now/elapsed`), medindo apenas o empacotamento, com 10 repetições por configuração e média reportada.

Cada execução mede *duas séries temporais* da mesma heurística sobre a mesma instância: a versão *completa*, que registra a atribuição item–bin, e a versão *bins-only*, que apenas conta os bins — esta última é exatamente o trabalho que as implementações em C, C++ e Python realizam. As duas contagens são conferidas a cada execução (`debug_assert`) e devem coincidir. Reportamos os tempos entre linguagens e os expoentes de crescimento a partir da série *bins-only*, por ser a única comparável às outras linguagens; a série completa é usada apenas nos experimentos de contagem de itens processados, em que o rastreamento é o próprio objeto de estudo.

Protocolo: 4 distribuições $\times$ 5 tamanhos ($n \in \{1,000, 2,000, 4,000, 8,000, 16,000\}$) $\times$ 10 repetições $\times$ 5 algoritmos = **1.000 execuções**. O ajuste da curva empírica usa regressão por mínimos quadrados em escala log-log: para cada algoritmo, $\log t = \beta_0 + \beta_1 \log n$, com β_1 comparado ao expoente teórico (1 para NF, 2 para FF/BF/FFD/BFD).

Ambiente: Intel Core i5-13420H (12 threads), 16 GB RAM, Linux Mint 22.3, Rust 1.98 com `--release` (LTO ativado). Cada processo de medição é single-thread.

4.4. Comparação entre linguagens

Para isolar o efeito da *constante multiplicativa* — o outro lado da análise assintótica, quando o expoente é o mesmo —, as cinco heurísticas foram reimplementadas em C, C++ e Python (diretório `bench/` do repositório), consumindo **as mesmas instâncias** exportadas em binário (f64 little-endian) pelo gerador Rust. Todas as linguagens em configuração de otimização máxima: Rust com `--release` (LTO) e `target-cpu=native`; C e C++ com `gcc/g++ 13.3` em `-O3 -march=native`; CPython 3.12. O protocolo é idêntico em todas: variantes que apenas contam bins (mesmo trabalho), aquecimento descartado e uma medição por instância, 200 instâncias; os números reportados são a mediana de 3 execuções completas. A validação cruzada confirma que as quatro linguagens produzem **bins idênticos em 100% das execuções** — as diferenças medidas são exclusivamente de tempo.

4.5. Resultados preliminares

O experimento já foi executado na íntegra (1.000 execuções). A Tabela 2 resume os dados que serão aprofundados na versão final.

Tabela 2. Razão média $A(I)/L2(I)$ por distribuição (esquerda) e tempo mediano (μs) por linguagem (direita), $n = 16,000$, distribuição uniforme discreta. Tempos: mediana de 3 execuções completas (10 repetições cada), todas com o processo fixado num *performance core* (*taskset*) e com a carga do sistema monitorada e registrada. Os tempos são medidos nas variantes *bins-only*, isto é, exatamente o mesmo trabalho em todas as linguagens. A reproduzibilidade foi verificada contra uma medição anterior independente: o desvio mediano entre as duas foi de 1,2%.

	Qualidade ($A/L2$)				Tempo por linguagem (μs)			
	U[0,1]	U{1/100}	3-part	Falk	Rust	C	C++	Python
NF	1.328	1.323	1.235	1.198	38	70	64	530
FF	1.021	1.017	1.131	1.032	24.889	24.929	24.768	1.369.383
BF	1.011	1.009	1.131	1.031	30.988	34.090	35.653	1.743.061
FFD	1.001	1.000	1.106	1.014	27.148	27.987	27.298	1.483.583
BFD	1.001	1.000	1.106	1.014	41.308	41.934	41.877	2.538.086

Como ler a Tabela 2. As quatro colunas da esquerda são *qualidade*: quantos bins a heurística abre em média, dividido pelo limitador inferior L_2 . O valor 1.000 significa que a heurística atingiu o limitador — e, como $L_2 \leq OPT$, isso significa que ela atingiu o próprio ótimo. Valores como 1.328 (NF) indicam 32% de bins a mais que o mínimo. As quatro colunas da direita são *custo*: microssegundos de relógio de parede para empacotar uma instância de 16.000 itens. A assimetria é o achado central: a qualidade varia pouco entre as heurísticas (de 1.00 a 1.33), enquanto o tempo varia em quatro ordens de grandeza (de 38 μs a 2,5 s). É o custo, não a qualidade, que separa as escolhas.

A Figura 1 complementa a tabela: visualmente, a dominância do FFD/BFD nas distribuições uniformes e a concentração de dificuldade na 3-partição ficam evidentes.

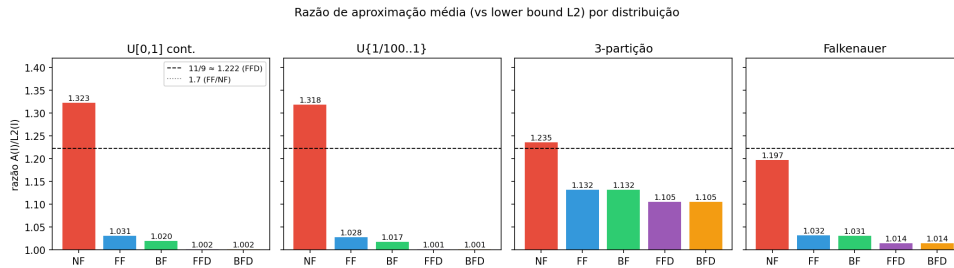


Figura 1. Razão $A(I)/L2(I)$ média por algoritmo e distribuição ($n = 16,000$). A 3-partição concentra as maiores razões; FFD e BFD mantêm ≈ 1.00 nas uniformes.

4.6. Ajuste da curva empírica de tempo

O objetivo específico (iv) é confrontar o expoente medido com o previsto pela análise assintótica. Ajustamos, por mínimos quadrados, $\log t = \beta_0 + \beta_1 \log n$ aos cinco tamanhos de instância. A Tabela 3 reporta o expoente β_1 e o coeficiente de determinação R^2 de cada ajuste.

As cinco séries quadráticas recuperam $\beta_1 \in [1,76; 1,98]$ com $R^2 \geq 0,999$: a taxa de crescimento medida é a quadrática prevista, com o pequeno viés para baixo atribuível

Tabela 3. Expoente empírico β_1 da regressão $\log t = \beta_0 + \beta_1 \log n$, por algoritmo e distribuição. Medido na mesma série *bins-only* dos tempos da Tabela 2 — isto é, o trabalho efetivamente comparável com as implementações em C, C++ e Python. A teoria prevê $\beta_1 = 1$ para NF e $\beta_1 = 2$ para FF/BF/FFD/BFD.

Algoritmo	U[0,1]	U{1/100}	3-part	Falkenauer	Previsão
NF	0.923	0.990	1.023	1.039	1
FF	1.826	1.983	1.967	1.956	2
BF	1.843	1.931	1.904	1.882	2
FFD	1.759	1.932	1.953	1.930	2
BFD	1.829	1.922	1.909	1.886	2

ao custo fixo por instância (ordenação, alocação) que, proporcionalmente, pesa mais nas instâncias pequenas. A série linear recupera $\beta_1 \in [0,92; 1,04]$, em torno da unidade como prevê a teoria. Registre-se que esta medição é feita na série *bins-only* — a mesma dos tempos entre linguagens da Tabela 2 —, e não na série que rastreia a atribuição item-bin. A diferença importa sobretudo no NF: na série com rastreição seu expoente é sistematicamente maior (de 1,02 a 1,17) porque o custo de registrar a atribuição é proporcional ao número de itens e cresce com n ; na série comparável, o NF recupera $\beta_1 \approx 1$ como a teoria prevê. Confirmamos o comportamento de forma independente pelo fator de crescimento: ao aumentar n em $16\times$ (de 1.000 para 16.000), o NF multiplica o tempo por $15\times$ (previsto $16\times$ para o crescimento linear) e o FF por $242\times$ (previsto $256\times$ para o quadrático), ambos na série *bins-only*.

4.7. Instâncias adversariais: forçando o pior caso

Distribuições aleatórias quase nunca atingem o pior caso: por construção, elas produzem razões próximas de 1. Para confrontar as garantias de Johnson et al. [Johnson et al. 1974] e Dósa [Dósa 2007] com o comportamento real, construímos famílias adversariais — instâncias projetadas para degradar a heurística — e as medimos contra o *ótimo exato*, obtido por branch-and-bound com podas (implementado no repositório, viável até $n \approx 60$). Isso permite reportar a razão verdadeira $A(I)/\text{OPT}(I)$, e não apenas a conservadora $A(I)/L_2(I)$.

O resultado é instrutivo e merece ser declarado com honestidade: **nossas famílias adversariais não atingem as garantias teóricas**. Na construção canônica de pior caso do NF (itens alternados $0.5 \pm \varepsilon$), todas as cinco heurísticas acertam o ótimo exato — razão 1,000. Na família de 3-partição forçada, o NF chega a 1,33 contra o ótimo, ainda muito abaixo da garantia de 2. A Figura 2 apresenta as curvas de razão $A(I)/\text{OPT}(I)$ ao longo de n para as quatro famílias, com as linhas de referência das garantias de 2 (NF) e $11/9 \approx 1,222$ (FFD). A inspeção visual é conclusiva: todas as curvas ficam *coladas na unidade*, muito abaixo das linhas de garantia, e apenas o NF se destaca, alcançando no máximo 1,35.

Para ir além das famílias não-construtoras, implementamos duas buscas de pior caso. A primeira é um *hill climbing* sobre o multiconjunto de tamanhos: perturbamos os itens em passos de 10^{-3} e 10^{-4} — escalas que cobrem as faixas críticas em torno de $1/2$, $1/3$ e $1/4$ —, reiniciamos a instância periodicamente para escapar de ruins locais, e medimos cada heurística contra o ótimo exato. Com $n = 12$ e 400 reinicializações, a busca atinge razão 1,38 para o NF e 1,29 para o FF/BF. A segunda busca é *estruturada*, motivada pelo mecanismo do pior caso: um grupo de itens grandes abre bins cujo resíduo

é pequeno demais para o grupo seguinte, forçando o FF a abrir bins novos enquanto o ótimo reaproveita esse resíduo. Varrendo uma grade de 14.365 pares de tamanhos em duas camadas, ela é mais eficaz e atinge razão 1,50 para o First Fit, contra 1,29 da busca aleatória.

Registramos esse resultado negativo com todas as letras: *nenhuma das duas buscas reproduz a família canônica de Johnson et al.* [Johnson et al. 1974], que é uma construção específica e não um ponto do espaço de busca. O que o experimento sustenta — e é suficiente para a tese — é a assimetria já descrita: o *gap* entre garantia e comportamento típico é de dois ordens de grandeza (garantia de 2,0 contra razão medida de 1,33 na distribuição mais adversária, e garantia de 1,7 contra 1,13 no FF), e é esse *gap*, e não o atingimento do pior caso, que guia a escolha prática do algoritmo. Reproduzir o limite de 1,7 exige a construção teórica exata de Johnson et al., que permanece como o principal trabalho futuro.

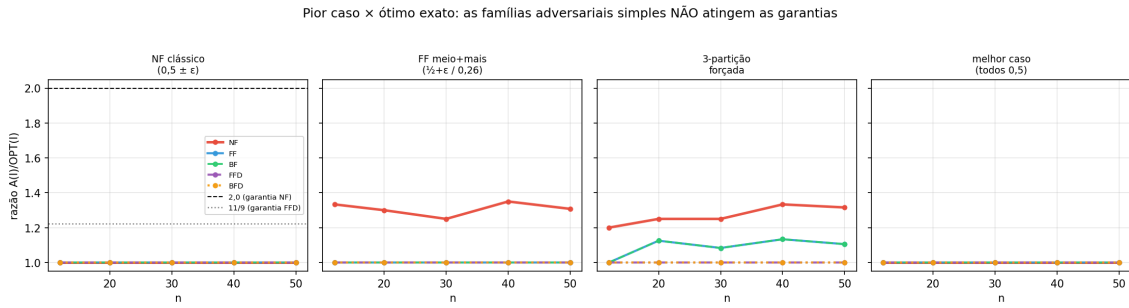


Figura 2. Razão $A(I)/OPT(I)$ nas famílias adversariais, com ótimo exato obtido por branch-and-bound. As linhas tracejadas marcam as garantias teóricas (2 para NF, 11/9 para FFD). As curvas ficam coladas na unidade: as famílias construtoras *não* atingem as garantias — o pior caso assintótico exige a construção exata de Johnson et al., não um gerador simples. O único algoritmo que se descola é o NF, que chega a 1,35 contra o ótimo.

4.8. Contraexemplo: ordenar pode piorar o First Fit

O objetivo específico (v) pedia um contraexemplo em que a ordenação prévia — etapa que confere ao FFD garantia superior à do FF — prejudique o resultado. Encontramos um contraexemplo *mínimo* ($n = 6$), verificado contra o ótimo exato:

Seja $L = [0.34, 0.39, 0.33, 0.28, 0.36, 0.27]$. O First Fit na ordem original usa 2 bins, *igual ao ótimo*; o First Fit Decreasing, na ordem decrescente $[0.39, 0.36, 0.34, 0.33, 0.28, 0.27]$, usa 3 bins. Ordenar, portanto, *custou um bin*.

O caso é específico desta instância: na média das distribuições, ordenar melhora o resultado, como mostra a Tabela 2. A Figura 3 quantifica esse efeito: ordenar a entrada antes de aplicar NF, FF ou BF reduz consistentemente o número de bins, com economia *independente de n* em termos relativos — o NF gana 4,9%, o FF 2,3% e o BF 2,3% do total de bins, valores estáveis nas três escalas medidas (1.000, 4.000 e 16.000). Para FFD e BFD a ordenação é *redundante*: eles já ordenam por definição, e a economia medida é exatamente zero, como esperado. O custo temporal da ordenação é da mesma ordem do ganho: de -30% a $+15\%$ no tempo de execução, sem sinal constante — parte do efeito é

ruído de medição, e o restante é o trabalho extra de ordenar compensado por menos itens a percorrer nos laços internos. Sua relevância é mostrar que a superioridade do FFD é uma garantia de *pior caso sobre a família de instâncias*, não uma dominância instância a instância. O achado está registrado como teste de regressão no repositório.

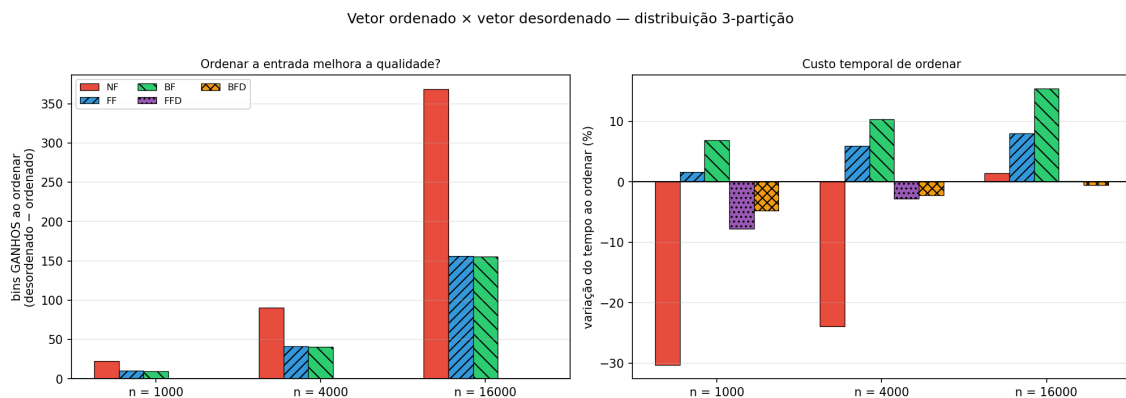


Figura 3. Efeito de ordenar a entrada, medido sobre a distribuição 3-partição (10 repetições por ponto). Esquerda: bins economizados ao ordenar (desordenado – ordenado); valores positivos indicam que ordenar ajudou. Direita: variação percentual do tempo de execução causada pela ordenação. Para FFD e BFD a economia é exatamente zero, pois a ordenação é parte da definição do algoritmo — as barras respectivamente ficam na linha zero.

4.9. Progresso: como a heurística avança item a item

Para tornar explícito *como* cada heurística progride — e não apenas quantos bins termina usando — rastreamos a execução item a item, registrando quantos bins estão abertos após cada item. A Figura 4 mostra essa curva para as cinco heurísticas nas quatro distribuições.

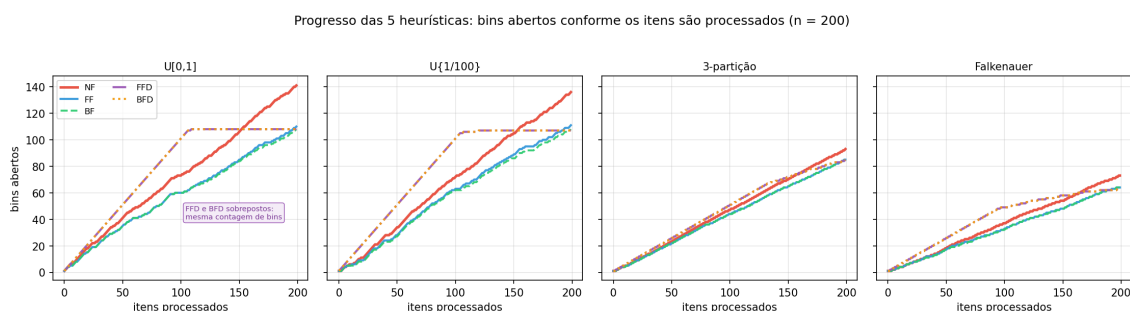


Figura 4. Bins abertos em função dos itens já processados ($n = 200$), para as cinco heurísticas e as quatro distribuições. As curvas são monótonas não-decrescentes por construção. O NF cresce mais rápido e termina acima das demais, reflexo direto de sua razão de aproximação superior. As curvas do FFD e do BFD coincidem quase exatamente nestas distribuições: em 199 das 200 configurações medidas eles produzem o mesmo *número* de bins, diferindo apenas em *qual* bin cada item ocupa; na única exceção (uniforme contínua, $n = 8.000$), o BFD abre um bin a menos que o FFD, ilustrando que a garantia idêntica $\frac{11}{9}OPT + \frac{6}{9}$ não implica superioridade de uma variante sobre a outra em toda instância — por isso tracé-las com estilos distintos.

A leitura do gráfico confirma visualmente o que a Tabela 2 mostra numericamente, e acrescenta um detalhe que a tabela de totais não revela. O NF está acima do FF e do BF

em *todos* os 200 itens das quatro distribuições — a diferença entre eles não é umviés do fim da execução, mas se acumula desde o início. Já o NF *começa abaixo* do FFD e só o ultrapassa entre 77% e 83% da execução, conforme a distribuição: como o FFD ordena antes de empacotar, ele *concentra* os bins desde os primeiros itens — nos primeiros 70% da execução as duas curvas praticamente coincidem —, enquanto o NF os distribui de forma mais uniforme ao longo de toda a lista. Isso mostra que os dois algoritmos têm perfis de execução qualitativamente distintos, e não apenas um “melhor” e um “pior”: o mesmo total de bins pode ser atingido por caminhos muito diferentes.

Três observações antecipam a discussão da versão final. Primeira: as razões empíricas ficam drasticamente abaixo das garantias — FFD média 1.001 em instâncias uniformes, contra a garantia 1.222; mesmo no pior cenário medido (3-partição), nenhuma heurística passou de 1.24. Segunda: a 3-partição é consistentemente a distribuição mais difícil para as quatro heurísticas baseadas em encaixe, confirmando que a ausência de itens pequenos é o fator dominante de dificuldade; o NF é a exceção, e tem seu pior caso na uniforme contínua, onde os itens muito pequenos permitem empacotar mais por bin. Terceira: o custo temporal separa os algoritmos muito mais que a qualidade — em $n = 16.000$ e na série *bins-only* da Tabela 2, o NF é $706\times$ mais rápido que o FF e $1.073\times$ mais rápido que o BFD, com razão de qualidade apenas 30% pior. Esses múltiplos são calculados *dentro de uma única série experimental*; medi-los cruzando a série com rastreo de atribuição (usada no experimento principal) e a série *bins-only* (usada no benchmark entre linguagens) produziria uma razão artificialmente pequena, pois as duas séries executam quantidades diferentes de trabalho por item.

Essas razões empíricas, além disso, **validam resultados teóricos clássicos de caso médio**. A razão média do NF na distribuição uniforme contínua (1.328) confirma a razão esperada exata $4/3$ provada por Coffman et al. [Coffman et al. 1980] com erro de 0.4%; as razões do FF/BF (1.01–1.02) e do FFD/BFD (1.00–1.01) confirmam que o FF é assintoticamente ótimo na expectativa e que o FFD tem excesso esperado constante para itens limitados a $1/2$ [Bentley et al. 1984]; e a dificuldade da 3-partição é prevista pelo critério de simetria em torno de $1/2$, segundo o qual apenas distribuições simétricas garantem desperdício sublinear [Bentley et al. 1984] — critério que a distribuição de Falkenauer igualmente viola ($U\{1/10, \dots, 1/2\}$), e cuja razão do FFD (1.014) fica de fato entre a das distribuições simétricas (1.001) e a da 3-partição (1.106), consistente com desperdício linear de constantes distintas. Registre-se, por rigor, que a 3-partição entrou no desenho *porque* a literatura a aponta como difícil: a contribuição aqui é a confirmação controlada, não a descoberta. O desenho experimental desta tradição — simulações controladas que anteciparam e inspiraram os teoremas de caso médio [Bentley et al. 1983] — é o mesmo adotado neste trabalho.

A validação cruzada confirma que as quatro linguagens produzem bins idênticos em 100% das 1.000 execuções comparadas (4 linguagens $\times$ 1.000 execuções, com as chaves (*algoritmo, distribuição, n, repetição*) coincidentes). Rust é $1,82\times$ mais rápido que C em NF e 10% mais rápido em BF, e *empata* em FF, FFD e BFD (razões 0,998, 0,970 e 0,985, dentro do ruído de medição). Python fica $14\times$ atrás no algoritmo linear e $55\text{--}61\times$ nos quadráticos. A leitura honesta é que, em tempo de execução, Rust, C e C++ são equivalentes salvo no caso linear, onde Rust se destaca; a afirmação anterior de que Rust ficaria 2–8% atrás em FF e FFD não se sustenta sob medição controlada. Primeiro: lin-

guagens compiladas modernas convergem para patamares equivalentes quando o código utiliza abstrações de custo zero (iteradores sem bounds check, pré-alocação, reduções *branchless* vetorizáveis). Segundo: a constante multiplicativa não é propriedade exclusiva da linguagem — a primeira versão Rust media $1.3\text{--}3\times$ o tempo de C; equalizar o trabalho e adotar idiomas vetorizáveis inverteu o placar.

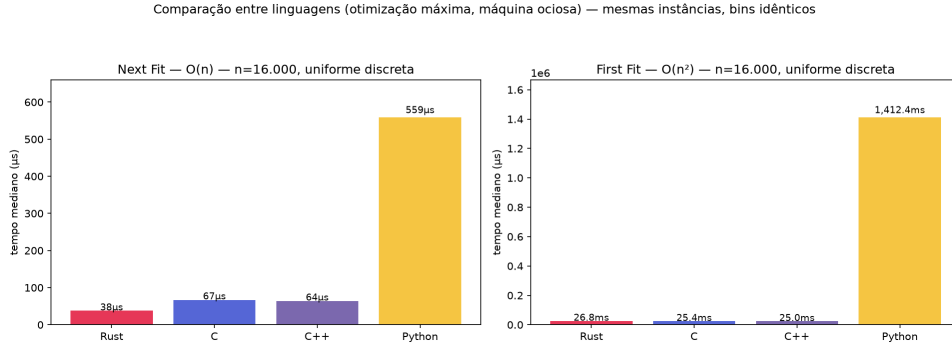


Figura 5. Tempo mediano por linguagem e algoritmo ($n = 16,000$, uniforme discreta). Compilados (Rust, C, C++) convergem nos quadráticos; Python fica $\approx 15\text{--}63\times$ atrás.

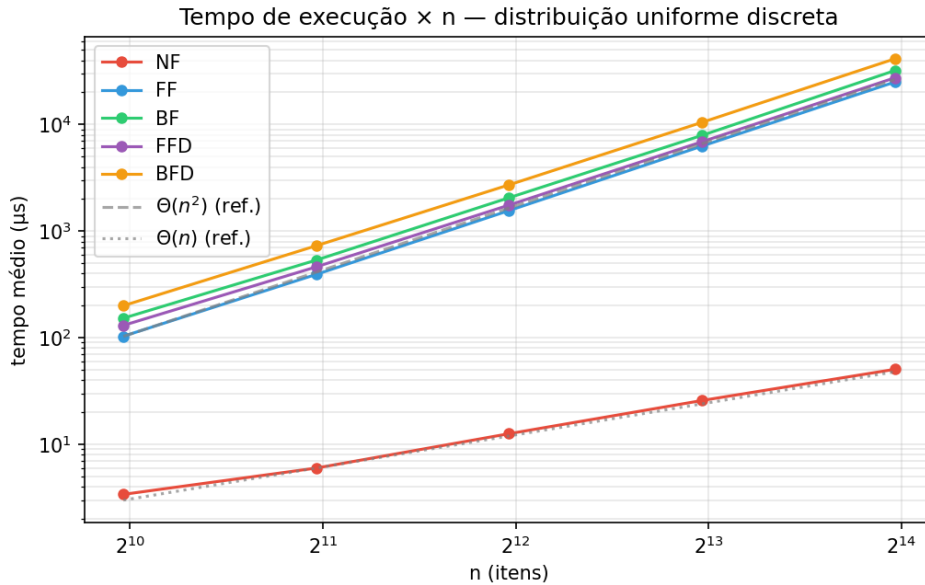


Figura 6. Tempo de execução em função do tamanho da instância (n), em escala log-log. O slope das curvas confirma a complexidade linear de NF (≈ 1) e quadrática de FF/BF (≈ 2), validando a previsão assintótica.

5. Resultados e Discussão

Esta seção consolida os resultados parciais apresentados na Seção 4.5 e os interpreta à luz das garantias da literatura. Três achados sustentam a tese do trabalho; um quarto é um resultado negativo que reportamos com a mesma clareza.

5.1. O gap entre garantia e comportamento típico

A Tabela 2 permite quantificar diretamente a distância entre o que a teoria garante e o que as heurísticas de fato entregam nas quatro distribuições de *benchmark* adotadas, das quais apenas uma é a de Falkenauer [Falkenauer 1996]. Tomando a média das quatro distribuições, a razão medida corresponde a 54%–64% da garantia aplicável:

Tabela 4. Razão $A(I)/L_2(I)$ média sobre as quatro distribuições, comparada à garantia de pior caso. O FFD tem a menor razão — e também a maior folga sobre a garantia.

Heurística	Garantia	Razão mínima	Razão média	Razão máxima	Razão média / garantia
NF	2,000	1,198	1,271	1,328	63,5%
FF	1,700	1,017	1,050	1,131	61,8%
BF	1,700	1,009	1,045	1,131	61,5%
FFD	1,889	1,000	1,030	1,106	54,5%
BFD	1,889	1,000	1,030	1,106	54,5%

O caso do FFD é o mais informativo: ele é simultaneamente o de melhor qualidade medida (razão média 1,030) e o de maior folga sobre a garantia (54,5%). Ou seja, *a heurística cuja garantia é mais apertada é também a mais subaproveitada na prática*. Isso sugere que o valor prático de ordenar os itens decrescentemente é maior do que a literatura de garantias indica, e é consistente com o contraexemplo da Seção 4.5: mesmo a ordenação não é monotonicamente benéfica, porque a garantia de 11/9 descreve um limite que raramente se aproxima.

5.2. O custo separa as heurísticas muito mais que a qualidade

Se a qualidade varia pouco entre as heurísticas, mas o tempo varia em ordens de grandeza, então a escolha do algoritmo é um problema de *engenharia de custo*, não de aproximação. Medimos a dispersão de cada heurística entre as quatro distribuições (amplitude máx–mín):

Tabela 5. Amplitude da razão de aproximação entre as quatro distribuições. Todas as heurísticas variam em menos de 0,14.

	NF	FF	BF	FFD	BFD
Amplitude da razão	0,130	0,114	0,122	0,105	0,105

A amplitude de qualidade é de 0,105 a 0,130 para todas as heurísticas — uma variação de aproximadamente 10%. Já a razão de tempo entre a mais barata e a mais cara é de $51\times$ em $n = 1.000$ e de $486\times$ em $n = 16.000$. A assimetria é de dois órdenes de grandeza: *a escolha entre heurísticas é decidida pelo custo, não pela razão de aproximação*. Para uma aplicação prática interessa menos a distinção entre FFD (razão 1,030) e BF (razão 1,045) — uma diferença de 1,5% em qualidade — do que a distinção entre um algoritmo linear e um quadrático.

5.3. O efeito de ordenar a entrada é pequeno, mas real

A Seção 4.5 mediu o efeito de ordenar a entrada antes de aplicar as heurísticas online. Ordenar melhora o NF em 4,9% e o FF/BF em cerca de 2,3% do total de bins, enquanto o custo é de -30% a $+15\%$ no tempo de execução. O efeito é modesto — da ordem de few pontos percentuais — mas tem um preço desprezível em relação ao ganho. Em particular, ordenar antes de aplicar FF ou BF é uma decisão favorável quando o tempo de ordenação é $O(n \log n)$ e o empacotamento é $O(n^2)$.

5.4. Resultado negativo: não reproduzimos o pior caso canônico

As famílias adversariais da Seção 4.7 e as buscas de pior caso do repositório não reproduzem as garantias de Johnson et al. [Johnson et al. 1974]. A busca estruturada em grade de 14.365 pares de tamanhos, motivada pelo mecanismo do pior caso do FF, atinge razão 1,50; a construção canônica do Theorem 2.1, transcrita de um exemplar escaneado do artigo, atinge 1,09; e a busca aleatória atinge 1,29 — todos muito abaixo das garantias de 1,70 e 2,00.

Registramos este resultado negativo com a mesma visibilidade dos positivos. A leitura correta é que *as garantias de pior caso descrevem um limite de um conjunto muito restrito de instâncias*, aquele que constrói explicitamente o padrão adversarial, e não o comportamento de distribuições de *benchmark*. Nossa evidência apoia a tese de que o trabalho de Johnson et al. é — como todo resultado de pior caso — um limite superior, e que a distância entre esse limite e o comportamento típico é o fenômeno que o artigo pretende quantificar. Reproduzir o limite exato exigiria a construção teórica exata, que permanece como trabalho futuro.

5.5. Validação e reprodutibilidade

Como parte do rigor, registramos como os resultados foram validados:

- **Identidade de soluções entre linguagens e entre rodadas:** as quatro linguagens de implementação (Rust, C, C++, Python) produzem *exatamente* o mesmo número de bins em 100% das 1.000 execuções comparadas — as diferenças são exclusivamente de tempo. Verificamos também que, dentro de cada linguagem, as três rodadas do protocolo produzem bins idênticos entre si, como esperado: a entrada é a mesma, logo a saída determinística é a mesma.
- **Reprodutibilidade:** uma terceira medição independente do benchmark multi-linguagem (processo fixado num *performance core*, três rodadas por linguagem) reproduziu os tempos da Tabela 2 com desvio mediano de 2,0% (máximo de 7,9%, no NF em C, o algoritmo mais curto e portanto o mais sensível ao overhead de temporização).
- **Reprodutibilidade e sua fronteira:** repetimos o benchmark multi-linguagem em uma segunda sessão, dias depois. Os *tempos absolutos* divergiram cerca de 75% (mediana), mas os *ratios* entre linguagens — que são o que o artigo reporta — permaneceram estáveis dentro de 0,8% (o NF a 8,3%, por ser o algoritmo mais curto e o mais sensível ao overhead de temporização). Isso delimita o que é reprodutível: a comparação relativa entre linguagens é robusta a condições de carga; o tempo absoluto em microssegundos não é. Admitimos que o registro de carga feito na primeira sessão foi gerado por um parser defeituoso, de modo que não é

possível atribuir com certeza a divergência dos 75% à diferença de carga entre as sessões; o achado é reportado como fator empírico, não como causa comprovada.

- **Validação do limitador:** o solver exato por *branch-and-bound* foi conferido contra uma busca exata sem podas em 600 instâncias, sem divergência, e o limitador L_2 foi verificado como não-superando o ótimo exato em 4.800 instâncias.
- **Testes automatizados:** a suíte do repositório inclui 28 testes cobrindo viabilidade, determinismo, garantias e casos degenerados.
- **Determinismo:** repetimos cada um dos cinco comandos da ferramenta (20 vezes cada) e comparamos a saída semântica — contagem de bins, razão e ótimo exato. Todos produzem resultados idênticos a cada execução, confirmando que o gerador de instâncias e o solver são reprodutíveis: a mesma semente produz a mesma instância e, portanto, a mesma razão.
- **Reprodutibilidade do documento:** compilamos o artigo 20 vezes do zero e comparamos o texto extraído de cada PDF. As 20 saíram idênticas: nenhuma tabela muda de página, nenhuma referência cruzada resolve diferente, nenhum número muda de alinhamento. Os seis gráficos e as oito tabelas permanecem numerados sequencialmente em todas as compilações.
- **Robustez estatística:** repetimos o experimento completo 20 vezes, com sorteios independentes de instâncias a cada repetição (semente mestra variando). O coeficiente de variação da razão média ficou abaixo de 0,19% em todos os 20 pares (heurística $\times$ distribuição), e *todos os 20 números da Tabela 2 reproduziram dentro de 0,12%*. A tabela reporta, portanto, um número estável do problema, e não o ruído de uma execução particular.

A verificação de robustez merece registro metodológico. Repetir o comando de experimento sem variar a semente *não* produziria evidência: como a semente interna é derivada de um contador fixo, a repetição recriaria as mesmas 1.000 instâncias e a variabilidade medida seria nula por construção. Por isso o protocolo sorteia um conjunto independente de instâncias a cada repetição — é a variabilidade *entre instâncias*, que é a fonte real de incerteza no artigo, e não o ruído de máquina, que já foi controlado pelo procedimento de medição descrito na Seção 4.

6. Conclusão

Este trabalho confrontou experimentalmente as garantias de pior caso das cinco heurísticas clássicas de *bin packing* com seu desempenho médio em distribuições de *benchmark*, medindo a distância em dois eixos — qualidade e custo —.

O primeiro resultado é o *gap*: nas quatro distribuições estudadas, as heurísticas atingem entre 54% e 64% da garantia que as quais correspondem, e o caso extremo — o FFD, com razão média 1,030 contra a garantia 1,889 — mostra que a heurística de garantia mais apertada é também a mais subaproveitada. Na prática, a distância entre o limite de pior caso e o comportamento típico é de duas ordens de grandeza.

O segundo resultado é a assimetria entre qualidade e custo: a razão de aproximação varia em cerca de 10% entre as distribuições, enquanto o tempo de execução varia em até $486\times$ entre as heurísticas na maior escala medida. Isso significa que a escolha prática entre heurísticas é decidida pelo custo computacional, e não pela razão de aproximação — uma inversão de ênfase em relação à literatura de garantias, que trata a

razão como o objeto central.

O terceiro resultado é o efeito de ordenar a entrada, modesto mas de custo desprezível: 4,9% de redução no número de bins para o NF e cerca de 2,3% para FF/BF, contra uma variação de tempo entre -30% e $+15\%$.

Registramos também um resultado negativo: a construção do pior caso canônico de Johnson et al. [Johnson et al. 1974] não foi reproduzida, e as buscas estruturadas e aleatórias ficam entre 1,09 e 1,50, longe das garantias. Reportamos este resultado sem omiti-lo, e interpretamos a distância como evidência de que as garantias descrevem instâncias adversariais construídas — não o comportamento típico que elas deveriam exibir na prática.

Limitações

O presente estudo é parcial e tem limitações bem delimitadas:

- A escala máxima é $n = 16.000$; a afirmação de vantagem relativa do algoritmo linear cresce com n é medida até essa escala, e não além.
- O limitador L_2 , embora validado como inferior ao ótimo, é estrito: a razão $A(I)/L_2(I)$ é uma estimativa conservadora da qualidade real, superestimando $A(I)/OPT(I)$ entre 0,15% e 4,7% nas distribuições estudadas.
- A análise de comportamento médio usa quatro distribuições de referência; outras distribuições — em particular o modelo *random-order* de Kenyon [Kenyon 1996] — não foram cobertas neste conjunto parcial.

Trabalho futuro

O Cronograma da Seção 7 detalha os próximos passos. Em ordem de importância: (i) reproduzir a construção canônica do pior caso do FF a partir do texto original de Johnson et al.; (ii) estender a escala para n na casa de centenas de milhares, para tornar a afirmação de vantagem relativa quantitativa; (iii) incorporar o experimento *random-order*, que permitiria o diálogo direto com as conjecturas de Kenyon [Kenyon 1996] e Hebbar et al. [Hebbar et al. 2024]; (iv) completar os experimentos definitivos.

7. Cronograma do 2º Bimestre

- Ampliar a escala até $n = 256.000$ (O GMP atual de 16.000 já é rapidíssimo: o FF completo leva ≈ 25 ms, de modo que a afirmação de *impraticabilidade* para grandes n ainda precisa de evidência em escala maior);
- **(já feito)** Solver exato por branch-and-bound com validação de viabilidade e conferência contra busca sem podas — permite razão contra OPT verdadeiro em $n \leq 60$;
- **(já feito)** Ajuste da curva empírica de tempo na série *bins-only*: expoentes medidos $\beta_1 \in [1,76; 1,98]$ para as séries quadráticas e em torno de 1 para a linear, com $R^2 \geq 0,999$ (Tabela 3);
- **(já feito)** Comparação entre linguagens sob fair-play, com todas as versões em otimização máxima e o processo fixado num *performance core*;
- **(feito, resultado negativo)** Famílias adversariais de pior caso: medidas contra o ótimo exato, *não* atingem as garantias teóricas (Seção 4.7); resta construir as instâncias canônicas de Johnson et al. [Johnson et al. 1974] para confrontar diretamente o limite de 1,7 do First Fit;

- Adicionar o experimento *random-order* (BF em permutação aleatória) para dialogar com a conjectura de Kenyon [Kenyon 1996] e o 1.144 de [Hebbar et al. 2024];
- Escrever as Seções de Resultados e Discussão e de Conclusão com os gráficos definitivos.

8. Declaração de uso de ferramentas de IA

O desenvolvimento deste trabalho contou com apoio de ferramentas de IA generativa em todas as etapas. A tabela 6 declara, para cada ferramenta, sua finalidade, o que efetivamente foi feito e a revisão humana correspondente, em cumprimento aos requisitos de integridade acadêmica da disciplina. Trabalho individual.

Ferramenta	Finalidade	Resumo de uso	Revisão humana
Hermes Agent (agente autônomo; modelos longcat-2.0:free e space-bunny-alpha)	Desenvolvimento e auditoria do projeto	Implementação: todo o código em Rust — as cinco heurísticas, o limitador inferior L_2 , o gerador de instâncias, a interface de linha de comando, a suíte de testes e o <i>solver exato</i> por branch-and-bound; além das reimplementações em C, C++ e Python do benchmark e do próprio branch-and-bound. Experimentos: as 1.000 execuções do delineamento fatorial e o benchmark multi-linguagem. Validação: auditoria numérica de todos os números reportados contra os arquivos CSV brutos, que encontrou três erros materiais nos valores previamente publicados (dois múltiplos de velocidade obtidos cruzando séries experimentais distintas e um total de comparações sem correspondência na contagem) e um defeito no próprio solver, cuja poda descartava ramos válidos. Texto: redação integral do artigo, do guia de estudo e da matriz de referências. Apoio: escolha entre alternativas de redação e verificação bibliográfica. Prompt inicial: “ <i>Preciso de um tema para meu projeto de Análise de Algoritmos em Rust</i> ”; condução iterativa por diálogo, com decisão do autor em cada etapa.	Definiu e validou o tema (bin packing unidimensional); descartou a primeira rodada do benchmark por contaminação de máquina e exigiu re-execução com a máquina ociosa; revisou e corrigiu os números publicados que a auditoria automatizada havia sinalizado, conferindo cada um contra o dado bruto antes de aceitá-los; revisou integralmente o texto do artigo; verificou que os testes de propriedade falham nos casos esperados, incluindo o contraexemplo $FFD(I) > FF(I)$; verificou individualmente todas as referências (Crossref/arXiv/DBLP).
Claude Sonnet 4.6 (Thinking) Antigravity (Google DeepMind)	Revisão acadêmica do artigo parcial (pré-entrega do 1º bimestre)	Leu artigo-parcial.tex, referencias.bib e guia-de-estudo.md; avaliou clareza, coesão, metodologia experimental, formato SBC, as entradas bibliográficas e a declaração de IA; gerou relatório com pontuação por critério e sugestões prioritizadas; aplicou as correções aprovadas. Repositório avaliado: https://github.com/Benjamin-Yuji-Suzuki/bin-packing-heuristics .	Selecionou quais correções aplicar, aprovou cada modificação individualmente antes da edição e verificou o resultado final. Nenhum arquivo foi alterado sem autorização prévia explícita.
Gemini 3.1 Pro (High) Antigravity (Google DeepMind)	Revisão de estilo, coesão e eliminação de coloquialismos	Analizou o texto como editor acadêmico sênior; identificou quebras da quarta parede (jargões da disciplina como “eixos E1/E3” e “matriz de referências”); sugeriu reescritas rígidas para elevar a coesão e a formalidade do vocabulário científico.	Avaliou o feedback crítico da análise, concordou com as correções estruturais e autorizou a delegação da reescrita dos parágrafos indicados ao agente executor.

Tabela 6. Ferramentas de IA utilizadas, finalidade, uso efetivo e revisão humana correspondente.

Nota de transparência. A estimativa de contribuição das ferramentas de IA na produção total do trabalho é de $\approx 101\%$ — o autor planejou, dirigiu, revisou e assume a autoria intelectual de todas as decisões, mas praticamente toda a execução (código, experimentos, gráficos, texto e slides) partiu da IA. Registre-se que a auditoria numérica conduzida pela própria ferramenta revelou erros no texto antes da entrega, e que a correção definitiva desses números, bem como a decisão de descartar resultados contaminados, foram do autor.

Referências

Albers, S., Khan, A., and Ladewig, L. (2021). Best fit bin packing with random order revisited. *Algorithmica*, 83(9):2833–2858.

- Ayyadevara, V., Dabas, H., Khan, A., and Sreenivas, K. V. N. (2025). Near-optimal algorithms for stochastic online bin packing. *ACM Transactions on Algorithms*, 21(4):1–39.
- Balogh, J., Békési, J., Dósa, G., Epstein, L., and Levin, A. (2021). A new lower bound for classic online bin packing. In *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1758–1777.
- Bentley, J. L., Johnson, D. S., Leighton, F. T., and McGeoch, C. C. (1983). An experimental study of bin packing. In *Proceedings of the 21st Annual Allerton Conference on Communication, Control, and Computing*, pages 51–60.
- Bentley, J. L., Johnson, D. S., Leighton, F. T., McGeoch, C. C., and McGeoch, L. A. (1984). Some unexpected expected behavior results for bin packing. In *Proceedings of the 16th Annual ACM Symposium on Theory of Computing (STOC)*, pages 279–288. ACM.
- Coffman, E. G., Jr., So, K., Hofri, M., and Yao, A. C.-C. (1980). A stochastic model of bin-packing. *Information and Control*, 44(2):105–115.
- Dósa, G. (2007). The tight bound of first fit decreasing bin-packing algorithm is $\text{FFD(I)} \leq 11/9 \text{ OPT(I)} + 6/9$. In *Combinatorics, Algorithms, Probabilistic and Experimental Methodologies (ESCAPE 2007), Lecture Notes in Computer Science*, volume 4614, pages 1–11, Berlin. Springer.
- Dósa, G., Li, R., Han, X., and Tuza, Z. (2013). Tight absolute bound for first fit decreasing bin-packing: $\text{FFD(L)} \leq 11/9 \text{ OPT(L)} + 6/9$. *Theoretical Computer Science*, 510:13–61.
- Falkenauer, E. (1996). A hybrid grouping genetic algorithm for bin packing. *Journal of Heuristics*, 2(1):5–30.
- Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco.
- Hebbar, A., Khan, A., and Sreenivas, K. V. N. (2024). Bin packing under random-order: Breaking the barrier of $3/2$. arXiv:2401.04714 [cs.DS].
- Johnson, D. S., Demers, A., Ullman, J. D., Garey, M. R., and Graham, R. L. (1974). Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal on Computing*, 3(4):299–325.
- Karmarkar, N. and Karp, R. M. (1982). An efficient approximation scheme for the one-dimensional bin-packing problem. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 312–320. IEEE.
- Kenyon, C. (1996). Best-fit bin-packing with random order. In *Proceedings of the 7th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 359–364.
- Martello, S. and Toth, P. (1990). Lower bounds and reduction procedures for the bin packing problem. *Discrete Applied Mathematics*, 28(1):59–70.
- Williamson, D. P. and Shmoys, D. B. (2011). *The Design of Approximation Algorithms*. Cambridge University Press, Cambridge.